

ADR-01: App de productividad all-in-one para estudiantes

ZenFlow — todo en un lugar, sin que nada te interrumpa

Campo	Valor
Autor	Dylan Vazquez
Fecha	15/05/2025
Estado	Propuesto

Contexto

La idea surgió de un problema que tengo yo mismo: cuando tengo que estudiar o hacer una tarea, termino usando tres o cuatro herramientas distintas al mismo tiempo. Google Calendar para ver qué tengo pendiente, Notion para tomar notas, un timer en el teléfono para el pomodoro, y aun así termino revisando Instagram a los diez minutos. El problema no es que me falten herramientas, es que tengo demasiadas y ninguna me obliga a quedarme concentrado.

ZenFlow es una aplicación de escritorio para Windows dirigida a estudiantes universitarios. La idea central es tener tareas, hábitos y un modo de enfoque en un solo lugar, donde el modo enfoque no sea solo un cronómetro bonito sino que realmente bloquee las aplicaciones que me distraen mientras estoy trabajando.

Las condiciones que me llevaron a tomar esta decisión son bastante concretas:

- Ya sé C# por el curso, así que no tengo que aprender un lenguaje nuevo encima de aprender a diseñar una arquitectura
- El semestre es corto, entonces no puedo arriesgarme con tecnologías que no conozco
- Trabajo solo en el proyecto, lo que significa que la arquitectura tiene que ser lo suficientemente clara para que yo mismo pueda mantenerla sin perderme
- Necesito acceso a APIs de Windows para bloquear procesos, y eso se hace mejor desde una app nativa de escritorio que desde una web

Decisión

Voy a construir ZenFlow como una aplicación de escritorio Windows usando C# con WPF en Visual Studio. La arquitectura va a seguir una separación en capas: la interfaz no va a saber nada de cómo se guardan los datos, y la lógica del negocio va a estar separada de ambas. Voy

a aplicar los principios que vimos en clase, especialmente SRP y DIP, para que agregar módulos nuevos (como el gestor de hábitos) no me obligue a tocar lo que ya funciona.

¿Por qué?

La razón principal es el modo enfoque. Para bloquear aplicaciones en Windows necesito usar APIs del sistema operativo como

EnumWindows o manejar procesos con System.Diagnostics.Process. Eso es C# puro, sin frameworks intermedios que lo compliquen. Si usara una app web tendría que recurrir a extensiones del navegador o herramientas externas, y ya no sería una sola app cohesionada sino un parche encima de otro parche.

La segunda razón es más honesta: WPF es lo que puedo aprender durante este semestre sin sacrificar la calidad de la arquitectura. Prefiero hacer algo bien con tecnología que conozco a hacer algo a medias con algo nuevo.

Alternativas consideradas

Alternativa	Por qué la descarté
Notion / Obsidian	Lo primero que pensé fue "¿para qué construir algo que ya existe?", pero el punto del proyecto es precisamente construirlo yo. Aparte, estas herramientas no me permiten bloquear otras aplicaciones del sistema, que es la funcionalidad que más me interesa.
App móvil con Flutter	No conozco Flutter ni Dart. Aprender un lenguaje nuevo al mismo tiempo que intento hacer una app completa en un semestre me parece demasiado riesgo. Prefiero aprovechar que ya sé C#.
App web (React + Node.js)	Misma razón que Flutter: stack desconocido. Además, una app web no puede bloquear procesos del sistema operativo tan fácilmente como una app de escritorio nativa, y eso es central en mi idea.

Consecuencias

✅ Lo que gano:

- Técnica: puedo implementar el bloqueo de aplicaciones directamente con APIs de Windows sin depender de nada externo. Eso hace que el modo enfoque sea una función real del sistema y no una simulación.
- Técnica: la arquitectura en capas me permite agregar el módulo de hábitos o de agenda sin tener que reescribir la lógica de tareas. Cada cosa vive en su lugar desde el principio.

- Proceso: al usar el mismo lenguaje y entorno del curso, puedo aplicar directamente los patrones que estamos viendo, lo que me sirve también para entenderlos mejor en la práctica.

⚠ Lo que sacrifico o asumo:

- Limitación técnica: la app solo va a funcionar en Windows. Si alguien con Mac quisiera usarla, no podría. Por ahora no es un problema porque yo mismo uso Windows, pero es algo que hay que tener claro desde el inicio.
- Deuda técnica: XAML tiene una curva de aprendizaje y es probable que la primera versión de la interfaz quede bastante básica. Si el proyecto sigue después del semestre, la UI va a necesitar trabajo.
- Riesgo: nunca he implementado el bloqueo de procesos en Windows, así que esa parte es la más incierta. Si resulta más complicado de lo esperado, podría tener que simplificar el modo enfoque a solo un cronómetro en la primera versión.

Diagrama

La arquitectura es en capas. La regla es simple: cada capa solo puede hablar con la de abajo, nunca se salta niveles ni habla hacia arriba.



Todavía no tengo código escrito, pero este es el esquema que voy a seguir. La separación entre repositorios y datos es importante porque si decido cambiar de JSON a SQLite más adelante, solo tengo que tocar esa capa sin que nada más se rompa.